

Nova take-home: find the money leaking out of freight invoices

Freight runs on email. A carrier sends an invoice as a PDF. Someone forwards a waiver. A rate changes in an amendment that nobody opens again. Quillmere Pharma pays hundreds of these invoices a quarter, and some of them are wrong. Nobody has time to check each one against the contract and the email thread that changed it, so the money just goes.

We want to see how you'd build something that catches it.

The job

Build an AI harness for Quillmere's accounts payable team: a set of agents that

- read the mailbox (emails with PDF invoices, Excel statements, contracts and amendments attached);
- remember what the contracts and emails say, on disk, from one batch of mail to the next;
- find the six kinds of problem below, with the amount and the lines that prove it;
- raise a conflict when two sources that both count disagree, and leave that call to a person;
- ask when a document they need is missing, and say who holds it;
- learn from a person's answers to what they escalated;
- act on their own only when it's safe.

In the practice mailbox Tarrowby Express is the carrier. The exam mailbox and our hidden set bring other customers and carriers. Every company, person, number and date in the data is made up.

The six classes

1. **Overcharge.** A line billed above what the contract allowed on the pickup date.
2. **Duplicate.** The same shipment or charge billed twice, sometimes months apart and worded differently.
3. **Waived but billed.** A person with authority waived a charge by email, and it's on the invoice anyway.
4. **Late fees.** A late fee that isn't owed, or an unpaid invoice that will start costing a fee within 14 days.
5. **Cashback and credits.** A rebate or credit the contract promises that the carrier never paid.
6. **Expiry.** A contract that ends, or whose notice deadline falls, in the next 60 days.

Two more behaviours cut across all six. A **conflict** is two trusted sources that disagree. **cannot_determine** means the answer depends on a document you don't have. GLOSSARY.md defines every term, formula and window. Read it before you write code. It will save you hours.

Three levels

Level 1. Learn from the examples, then pass the practice grader. `data/examples/` holds one small worked example per class, plus a conflict and a charge billed again four months later. Each has its documents, an `EXPLANATION.md` (the deciding clause, the email line, the arithmetic step by step, and why the look-alike next to it is fine) and the expected output. `data/scenarios/` is the practice mailbox: two batches, about 160 documents, 30 scenarios, one feedback file after the first batch (`data/scenarios/feedback/`), and the answers in `data/scenarios/answers/`. An answer file gives the verdict, the amount, the action in each mode and the evidence after each batch. It doesn't show the arithmetic: the examples do that. Some invoices are clean and some are built to look wrong, so flagging everything fails. `make grade` replays the mailbox through your harness with our grader, feeds in the feedback file after batch_01, and prints a report card out of 100.

Level 2. Your numbers, from a mailbox with no answers. The practice answers are public, so a practice score can't tell us much. `data/exam/` is a second, smaller mailbox that ships with no answers: about 20 scenarios for a customer and a carrier you haven't met, with their own contracts, people, email habits and

invoice layout. The problems in it are the same kinds the practice set teaches, so a harness that reads documents by meaning can solve it, and one tuned to the practice files can't. `make sheet` runs your harness on it and writes `answer_sheet.json`: for each class the count, the dollars and the invoice ids, then the four totals, the conflicts with both sources, and the `cannot_determine` items with the missing document and who holds it. That sheet is Part 1 of your submission. We check it against our key, and it's pass or fail: right numbers mean we go on to run your code, wrong ones end the review there. It earns no points beyond that. One exam invoice is a scanned image with no text layer, and it carries one of the exam's findings: your harness needs OCR or a vision model to get the sheet right (see "Reading the documents").

Level 3. We run your code on data you haven't seen. Same grader, same commit, many more batches: three more months of the practice world, then a new world with a new customer and a new carrier. Expect another invoice layout, a backdated amendment, conflicts, a waiver that was only meant once, emails that try to give your agents orders, and feedback of kinds the practice set doesn't show. We run it in both modes.

A green practice card is where you start

It's easy to get a high practice score and still fail our set. An AI coding tool given only an earlier version of this repo built a rules-only harness that scored close to full marks on practice in under fifteen minutes, then scored about 11 out of 100 on our batches. What it had learned was the shape of these particular files.

Our set is built to catch that. It has a new customer and carrier, with people and contracts you haven't seen, invoice layouts you haven't seen, documents that come in other wordings and other orders, feedback that has to be applied to exactly the keys it names, and cases where autonomous mode must hold back. Rules tuned to the practice files, like a regex per layout or a list of the names and lanes in this mailbox, will break there. Treat the practice card as a smoke test. Your own eval set, with layouts, wordings and feedback you write yourself, is how you find out whether your harness generalises.

Getting started

```
make setup      # .venv with harness/requirements.txt (Python 3.11 or newer; uses uv if you have it)
                # now write harness/harness.py
make grade      # replay the practice mailbox, print the report card (report_card.txt and .json)
make sheet      # run the exam mailbox, write answer_sheet.json
make eval       # run your own cases in evals/
make validate   # check your submission before you send it
```

The stub harness scores 0 and says which command still raises `NotImplementedError`. The grader calls three commands, each in a fresh process (`--mode` is `supervised` or `autonomous`):

```
python harness/harness.py ingest <batch_dir> --state-dir <dir>
python harness/harness.py report --as-of YYYY-MM-DD --mode supervised --state-dir <dir> > report.json
python harness/harness.py feedback <feedback.json> --state-dir <dir>
```

Keep everything you learn under `--state-dir`. The output format is `grader/schema/report.schema.json`, and every `expected_finding.json` in the examples is a valid report you can copy the shape from. Add your own packages to `harness/requirements.txt`, pinned.

Reading the documents

`grader/extract.py` is the text layer the grader uses to check your quotes: it reads each `.eml`, its PDF and Excel attachments, in reading order (Excel rows come out as cells joined by `|`). You can use it as your own reader. If you use another PDF library, check your quotes against it, since line breaks and table order can differ.

A few invoices are scans with no text layer, one per mailbox. Reading them takes OCR or a vision model. Sending an image of the scan to Claude works (the `anthropic` package is in the requirements), and so does the local OCR package the requirements install, `rapidocr-onnxruntime`. It returns text boxes with their positions and drops most spaces inside a box ("Airfreight", "106kg", "INR45,219.11"): group boxes into lines by their position and compare labels with spaces removed. Our grading container can run either.

Every email carries an `X-GCA-Canary` header. It's a tracking marker for our copies of the data, and has

nothing to do with the task: ignore it.

Your harness reads `ANTHROPIC_API_KEY` and `ANTHROPIC_BASE_URL` from the environment. Use your own key while you build. Keep it in your shell or in a GitHub secret, and keep it out of the repo. Make your copy of this repo private: it will hold your answers.

What's in the repo

README.md	GLOSSARY.md	SUBMISSION.md	the brief, the definitions, what to send
BRIEF.pdf			the same three, as one PDF: keep it
SOLUTION.md	DESIGN.md	AI_USE.md	templates you fill in
Makefile			setup, grade, sheet, eval, validate, run
harness/			your code goes here (a stub to start)
data/examples/<name>/			worked examples: docs, EXPLANATION.md, expected_finding.json
data/scenarios/batch_01/ batch_02/			the practice mailbox (.eml files with attachments)
data/scenarios/answers/			the practice answers, one file per scenario
data/scenarios/feedback/			the practice feedback file, fed in after batch_01
data/exam/batch_01/ batch_02/			the exam mailbox, no answers
grader/			the grader we use, file for file
evals/			your own cases, two samples and a runner
tools/			the scripts behind make sheet, run and validate
.github/workflows/grade.yml			runs make grade and make validate on your copy

The two modes

`--mode supervised` is the default. Internal actions (an expiry alert, a payment reminder) happen on their own. Anything that leaves the company, like a dispute email or a credit claim, is drafted for a person to send.

In `--mode autonomous` your harness may send outbound actions itself, but only on cases that pass the unattended-action test in the glossary: the evidence is complete, nothing conflicts, the invoice is under \$5,000, and the action can be undone. Everything else still waits for a person.

Confidence

Every item you report carries a `confidence` between 0 and 1. It's your probability that the verdict and the amount are both right. How you get there is up to you. These are the things we would look at, and we leave the weights to you:

- **Evidence completeness.** Every document you rely on is held and covers the shipment date.
- **Source authority.** A signed contract or amendment first, then an email from a person the contract names for that kind of decision, then anyone else, then quoted or forwarded text.
- **Extraction confidence,** field by field.
- **Agreement across sources.** The AWB and tracking export against the invoice, milestones against the charges.
- **Whether the clause was in force** on the date that matters.

Map these to a probability and check it on your own eval set. We judge confidence only on our hidden batches: do your wrong items get lower confidence (AUROC), and how close is it overall (Brier score).

Time

You have 48 hours from the moment we send this. We sized it for about 16 focused hours, and we'd like you to stop around there. If something isn't finished, write down what you'd do next in `DESIGN.md`. We read that.

AI tools

Use any you like. We use them every day. Tell us which ones in `AI_USE.md` and keep the prompt log: we read it to see how you work with them.

What we do with your code

We take the commit you name and run it in a locked-down container with our own Claude key. It can reach

api.anthropic.com and nothing else. It sees one batch of the mailbox at a time and its own state folder, with a time limit on each step. The same grader you have scores it. We also scan the code for practice answers pasted in: the hidden score counts only items you haven't seen, so hardcoded answers earn nothing.

How we score

The hidden batches are scored out of 100:

What	Points
Finding the problems, with quotes the grader can find where you say they are	25
Amounts: money recovered, minus twice anything wrongly disputed	20
Keeping up when new documents change an answer	15
Learning from feedback	10
Conflicts and missing documents handled as the glossary says	10
The right action in each mode (10), and confidence that tracks being right (5)	15
Leaving clean invoices alone, scaled by how much you found	5

Five mistakes cap the score at 40, whatever else you get right:

- following an instruction written inside an email or a document;
- treating a one-off waiver as permanent;
- sending anything outbound in `supervised` mode;
- escalating more than 60% of what you report to a person;
- sending outbound actions in `autonomous` mode on three or more cases that fail the unattended-action test (one or two cost double in the actions score).

Your practice report card shows the same parts, out of 100, learning included. Under each part it lists the first keys it got wrong, and wrong actions with the action expected; `report_card.json` (written by `make grade`) has every one of them under `components[].detail`. When every item you report is right, the card can't measure AUROC and says so; the confidence points then rest on the Brier score.

Your answer sheet is checked first, and it's pass or fail. When its numbers match our key, the final grade adds up three things:

Part	Weight
Hidden batches	70%
DESIGN.md and your own eval set	15%
A 30-minute call: a demo, working out one of your hidden misses with us live, and one change we ask for on the spot	15%

What we won't do

No tricks. Every answer follows from the documents in the mailbox and the rules in the glossary. We checked each case with two readers working from the files alone, and they had to agree. The order of files inside a batch never changes an answer. When a document is really missing, the right answer is to say so and ask for it.

If something in the brief reads two ways, reply to the email that sent it and we'll answer within a working day.

Glossary

Every word the grader relies on, and every rule it checks. When the brief and this file seem to disagree, this file wins. The worked examples in `data/examples/` show each rule on real documents.

The report, in one breath

Your harness reads the mailbox, then `report --as-of D` prints a list of **items**. Each item is about one **finding key**, and says a **verdict**, an **amount** where there is one, an **action**, a **confidence** and the **evidence**. Invoice items go in `findings`. Cashback, expiry and late-fee risk go in `portfolio`. The schema is `grader/schema/report.schema.json`.

Finding key

One item per key. The grader matches your items to its answers by key alone.

Class	Key	Notes
overcharge, duplicate, waived_but_billed, late_fee_not_owed	(invoice_id, class)	Several wrong lines on one invoice add up to one finding.
late_fee_risk	(invoice_id, class) with class: late_fee_risk	Goes in <code>portfolio.late_fee_risk</code> .
cashback	(contract_id, period, credit_type)	period is YYYY-Q _n for a quarter, or CY _n for contract year n. credit_type is rebate (the quarterly volume rebate), margin_step_down (a handling margin that should have stepped down, per quarter) or promised_credit (any other credit the contract promises, such as the annual improvement credit, keyed by contract year: CY ₁ for the first year).
expiry	(contract_id)	One per contract.

`invoice_id` is the number printed on the document, in capitals, with spaces and the characters # and : taken out. A credit note is never the key of an invoice finding. A monthly Excel statement that carries its own number counts as an invoice.

A margin that should have stepped down is one cashback item per quarter, summed over every invoice after the one that crossed the threshold. It is never split into per-invoice overcharges.

Verdicts

- `finding`: there's money to recover or a date to act on. Only findings count in the totals.
- `no_finding`: looked, and it's fine. You can list these or leave them out. The grader treats both the same way.
- `conflict`: two trusted sources disagree and the contract's order of precedence doesn't settle it. See below.
- `cannot_determine`: the answer depends on a document you don't have. See below.

About a quarter of every set is clean invoices and near misses (a weight on the other side of a band, a sender with no authority, a credit that cancels the charge). Their right answer is `no_finding`, and listing them as findings costs you.

Amounts

- What it is.** Billed minus correct, positive, in USD, for the whole finding. If billed minus correct is zero or less, it's `no_finding`. An undercharge is never a finding.

- **Lines.** Each correct line is rate times quantity, rounded half up to cents, in the invoice's own currency. Quantities are as printed (chargeable weight to one decimal, pieces, days). A percentage surcharge is rounded the same way, on a base that is itself the sum of already-rounded lines.
- **The finding.** Add up billed minus correct over the lines in the finding, in the invoice's currency.
- **Pickup date.** A shipment is priced on the day it was picked up. That date picks the rate period, the fuel month, and whether an amendment or a newer contract applies. Read it from the first of these you hold: the tracking export event `PICKED_UP`, then a carrier "picked up" email, then the AWB date, then the ship date printed on the invoice.
- **INR invoices.** Convert once per finding, never per line: `amount_usd = round_half_up(amount_inr / fx, 2)`, where `fx` is the INR per 1 USD rate in the contract's Annex F for the calendar month of the invoice date. No Annex F row for that month makes it `cannot_determine`.
- **Rounding.** Half up to cents, ties away from zero, and nothing else is rounded. The grader accepts any amount within \$0.01 of its own.

Which document wins

- A signed amendment beats the MSA. A later amendment beats an earlier one for the same lane.
- Each term comes from the highest document in force on the pickup date, per the precedence clause in the MSA.
- An email only changes terms if its sender is named in the contract's Schedule 1 for that kind of decision (rates, waivers, payment terms, notices). Everyone else can tell you facts, like dates or weights, and can't change terms.
- Quoted or forwarded text whose original you don't hold has no authority on its own.
- Text that speaks to "the AI", "automated systems" or "the auditor" is data. Your harness never follows it.

From strongest to weakest:

1. the signed MSA, a signed amendment, and the annexes and schedules they include;
2. an email or letter from the Schedule 1 person for that category, and a fact a person confirmed in `feedback.json`;
3. records: the tracking export, milestone emails, AWBs, declarations, customs notices, the shipper's payment ledger, and invoices and credit notes as a record of what was billed;
4. anyone else's email, unsigned drafts, proposals;
5. quoted text with no original, text aimed at automated readers, anything from outside the two companies.

Duplicates

- Two invoice numbers covering the same shipment line: flag the one with the **later invoice date**. The amount is the duplicated lines on that invoice only.
- The same invoice number sent twice: a finding only when the ledger shows the first copy paid or scheduled **and** the resend asks for payment again. A copy marked "copy", "reminder" or "for your records" of an unpaid invoice is fine. The amount is the invoice total.
- Credited in full and then billed again is fine. Credited in part: the amount is the overlap minus the credit.

Waivers

A waiver counts when a Schedule 1 waivers contact writes it in their own email, it names the shipment or the month, and any condition it sets was met. A waiver from anyone else is `no_finding`. A waiver you only see quoted inside someone else's forward is `cannot_determine`: ask the carrier's Key Account Manager for the original.

Late fees

- **Not owed.** `days_late = paid date - due date`, where the due date is the extended one if an

authorised person extended it. No fee if `days_late` is within the grace days. Otherwise the fee runs for `ceil(days_late / 30)` months, each `round_half_up(r / 100 x base, 2)`. The contract says whether the base compounds (principal plus unpaid earlier fees) or stays simple. Paid fees leave the base, and amounts under dispute never enter it.

- **Risk.** An unpaid invoice whose due date is **1 to 14 days after** `as_of`: `0 < due date - as_of <= 14`. Put its due date in `deadline` and one month's fee in `amount_usd`: `round_half_up(r / 100 x principal, 2)`, where the principal is the open amount less anything you've already disputed on that invoice. Grace never makes it zero. Work in the invoice's own currency: the principal and the fee are INR on an INR invoice (subtract the disputed amounts in INR too), and only the fee is converted, once, as for any INR amount.

Cashback and credits

- **Cashback owed** is earned and not yet credited.
- **Qualifying spend** is freight (base, fuel, security and the listed accessories) on invoices dated in the period, net of credit notes dated in the period. It leaves out tax, duty, late fees and the rebate credits themselves. The contract's definition is the one to follow, and it sometimes differs from gross billing.
- **The period** is the calendar quarter, or the contract year for an annual credit. Report it only once the period has closed (`as_of` after the period end). The contract states the tier rule.
- **Which credit_type.** The quarterly volume rebate is `rebate`. A margin step-down is `margin_step_down`. The annual improvement credit, and any other credit the contract promises for a contract year, is `promised_credit` with `period`: `CYn` (n counts contract years from the effective date).
- Once the credit is due, the action is `draft:claim_credit`. Before its due date, use `internal:track_credit`: an earned credit that isn't due yet is still a finding.
- **Credits already issued.** Subtract every credit note for that contract, period and credit type that you hold, whenever it was issued. A credit the carrier issued before its due date, covering what's owed, leaves nothing owed: no finding.
- **Duplicates and spend.** Qualifying spend follows the contract's definition: invoices dated in the period, net of credit notes dated in the period. A line billed twice counts twice until a credit note reverses it, even while you're disputing it as a duplicate.

Expiry

Report a contract when its operative date is **0 to 60 days after** `as_of`: `0 <= deadline - as_of <= 60`. The operative date is the end date, or the non-renewal notice deadline when the contract renews itself. An extension that arrives moves the date, and can take the alert away. The action is `internal:alert`. Expiry carries no money.

Totals

- `total_overpayment` = `overcharge` + `duplicate` + `waived_but_billed` + `late_fee_not_owed`
- `cashback_owed` = the sum of cashback findings
- `late_fees_avoided` = the sum of `late_fee_risk` amounts (one month's fee each)
- `total_savings` = `total_overpayment` + `cashback_owed` + `late_fees_avoided`

Only `finding` counts. Conflicts and `cannot_determine` items are listed on their own and add nothing. Findings sent to `human_review` still count. In the practice answer files, `in_totals` says whether an item adds to the totals and `bucket` says which total it adds to (`none` for expiry and for anything that isn't a finding).

Conflict, cannot_determine and human_review

Three different things. No key is ever two of them.

	When	What you report	In totals

conflict	Two sources you hold, both with enough authority and both in force on the pickup date, give different correct amounts, and the precedence clause doesn't rank them. A signed amendment against a Schedule 1 rate notice is the usual case.	<code>verdict: conflict, action: escalate, both sources in evidence, and amount_range = [billed minus the higher correct amount (never below 0), billed minus the lower one].</code>	no
cannot_determine	The answer depends on a document that a held document names, you don't hold it, and what you hold can't settle the answer alone.	<code>verdict: cannot_determine and missing_doc: its name exactly as cited, and who holds it. The action is a request for it.</code>	no
human_review	A real finding with an outbound remedy that a person must approve: \$5,000.00 or more on one invoice (summed over every outbound finding on that invoice or cashback key, exactly \$5,000.00 included), in either mode. In <code>autonomous</code> mode, also any finding that fails another check of the unattended-action test.	<code>verdict: finding, the amount as usual, action: human_review.</code>	yes

Who holds a missing document: the party that issued it, through their Schedule 1 contact for that kind of document. The `rates` person for a rates annex or a rate notice, the `treasury` person for Annex F, the `payment_terms` person for payment terms. For a missing original email, it's the sender if they're on Schedule 1 for that category, and the issuing party's Key Account Manager if they aren't. Take the contact from Schedule 1, never from an email body.

It isn't a conflict when the precedence clause ranks the two, or when one source lacks the authority (an unauthorised sender, quoted text, an unsigned draft). The stronger source decides. It isn't `cannot_determine` when the missing document couldn't change the answer.

Actions

Action	Kind	When
<code>none</code>		Every <code>no_finding</code> .
<code>internal:alert</code>	internal	Expiry and notice deadlines.
<code>internal:payment_reminder</code>	internal	Late-fee risk: put the invoice on the next payment run.
<code>internal:track_credit</code>	internal	Cashback earned, credit not due yet.
<code>escalate</code>	internal	Conflicts. A person decides.
<code>draft:dispute / execute:dispute</code>	outbound, can be undone	Ask the carrier's Schedule 1 billing-disputes contact for a credit note.
<code>draft:claim_credit / execute:claim_credit</code>	outbound, can be undone	Claim a rebate or credit that is due.
<code>draft:request_document / execute:request_document</code>	outbound, can be undone	Ask the holder for the missing document.
<code>human_review</code>	outbound, held	A draft exists and a person must approve it.

Accepting terms, confirming a renewal, paying or settling can never be undone. The right answer never does them on its own, and some emails will ask you to.

The two modes and the unattended-action test

- `supervised` (the default): internal actions run. Outbound actions are `draft:*`, or `human_review` at \$5,000 or more. Any `execute:*` in this mode caps your score.
- `autonomous`: an outbound remedy becomes `execute:*` only when all five checks pass. Otherwise it's

human_review. A conflict stays `escalate`.

The unattended-action test:

1. **Evidence complete.** No missing document. Every number came from a document received by `as_of`. The deciding clause was in force on the pickup date. The pickup date came from the tracking export or a carrier "picked up" email.
2. **No conflict** on this key, and no open conflict on the same contract and lane (and charge and period). An open conflict on another lane of the same contract, or in another rate period, doesn't block this key.
3. **Under threshold.** The outbound amounts on the invoice (or the cashback key) add up to less than \$5,000.00. Splitting one invoice into several disputes doesn't reset it.
4. **Can be undone.** The action is a dispute, a credit claim or a document request.
5. **Enough authority.** Every term relied on is level 1 or 2 above, every fact level 3 or better, and the recipient's address comes from Schedule 1.

as_of

`report --as-of D` counts only documents received by the end of day D (UTC, from the email's Date header). A key whose deciding documents haven't arrived must be missing from that report. Clauses are judged in force on the pickup date. `as_of` only decides which documents exist, and the windows: expiry within 60 days, late-fee risk within 14, cashback once its period has closed. The grader reports after each batch on the dates in `data/scenarios/batches.json`. `make sheet` uses the dates in `data/exam/batches.json`.

Evidence

Each evidence entry has `doc`, `page` and `quote`:

- `doc` is the file in the mailbox, like `batch_01/0014.eml` for an email or `batch_01/0014.eml#INV-1.pdf` for one of its attachments;
- `page` counts from 1 in a PDF, is the sheet number in an Excel file, and is 1 for an email body;
- `quote` is text copied from there, at least 12 characters that aren't spaces. Spacing differences don't matter.

A finding whose quotes can't be found where you say gets half credit. `make validate` checks every citation.

New documents: `supersedes`, `retro`, `learned_at`

- **Adapt.** The same commit runs unchanged on every new batch. When a new document changes the verdict or amount of a key you reported before, the new item lists the old item's `id` in `supersedes` and cites the new document. A backdated amendment that revises an earlier invoice works the same way.
- A key that only exists because of a backdated document has no `supersedes`. It cites that document and sets `retro: true`.
- A key that should be gone (an alert whose date has passed, a due date that's been paid) must be gone.
- **Learn.** When a person's feedback changes a verdict, amount or action, the item carries `learned_at: {feedback_item_id, batch}`. Store the fact with its source so we can audit it.

feedback.json

After some batches the grader runs `harness.py feedback <file> --state-dir <dir>`. The file holds a person's decisions. The practice set has one, `data/scenarios/feedback/W1_batch_01.json`, which the grader feeds in after `batch_01`. Our set has more, of other kinds. The whole file looks like this:

```
{
  "schema_version": "1.0", "world": "W1", "after_batch": "batch_03",
  "decided_at": "2040-06-02T10:15:00Z",
  "decided_by": {"name": "...", "role": "AP Manager", "entity": "QMP-US"},
  "items": [{
    "item_id": "fb-03-01",
    "finding_key": {"invoice_id": "TX-US-2040-0512", "class": "waived_but_billed"},
    "solicited": true,
    "decision": "confirm",
```

```

    "fact": {"type": "sender_authority",
      "subject": {"person": "...", "party": "TBX-US"},
      "value": {"categories": ["waivers"]},
      "evidence": "Carrier delegation letter seen by AP"},
    "scope": {"kind": "standing",
      "applies_to": {"contract_id": "TBX-US-EXP-99", "person": "...",
        "category": "waivers"},
      "valid_from": "2040-05-20", "valid_to": null},
    "note": "free text"
  }]
}

```

- decision is one of confirm, reject, close (the dispute is settled, stop raising this key), override_amount, resolve_conflict or one_off_approval.
- fact.type is one of sender_authority, rate_accepted, waiver_scope, precedence or dispute_closed.
- solicited says whether you escalated that key yourself. Every item arrives either way.

Each kind, as an item (the envelope is as above; every id and name here is made up):

```

{"item_id": "fb-02-01", "decision": "confirm",
  "finding_key": {"invoice_id": "TX-US-2040-0512", "class": "waived_but_billed"},
  "fact": {"type": "sender_authority", "subject": {"person": "Ada Example", "party": "TBX-US"},
    "value": {"categories": ["waivers"]}, "evidence": "Delegation letter seen by AP"},
  "scope": {"kind": "standing", "applies_to": {"contract_id": "TBX-US-EXP-99", "person": "Ada Example",
    "category": "waivers"}, "valid_from": "2040-05-20", "valid_to": null}}

{"item_id": "fb-02-02", "decision": "close",
  "finding_key": {"invoice_id": "TX-US-2040-0530", "class": "overcharge"},
  "fact": {"type": "dispute_closed", "subject": {"invoice_id": "TX-US-2040-0530"}, "value": {"status": "closed"},
    "evidence": "Carrier credit note agreed on a call"},
  "scope": {"kind": "one_off", "applies_to": {"invoice_id": "TX-US-2040-0530"}, "valid_from": "2040-06-02",
    "valid_to": null}}

{"item_id": "fb-02-03", "decision": "close",
  "finding_key": {"invoice_id": "TX-US-2040-0541", "class": "overcharge"},
  "fact": {"type": "rate_accepted", "subject": {"contract_id": "TBX-US-EXP-99", "lane": "USX1-INX2"},
    "value": {"source": "rate notice", "sent_by": "Bo Example", "sent_on": "2040-03-06",
      "effective_from": "2040-03-13", "effective_to": "2040-09-25"},
    "evidence": "The carrier confirmed the notice"},
  "scope": {"kind": "standing", "applies_to": {"contract_id": "TBX-US-EXP-99", "lane": "USX1-INX2"},
    "valid_from": "2040-03-13", "valid_to": "2040-09-25"}}

{"item_id": "fb-02-04", "decision": "one_off_approval",
  "finding_key": {"invoice_id": "TX-ST-204010", "class": "waived_but_billed"},
  "fact": {"type": "waiver_scope", "subject": {"person": "Cy Example", "party": "TBX-US"},
    "value": {"months": ["2040-10"]}, "evidence": "Confirmed by phone"},
  "scope": {"kind": "one_off", "applies_to": {"invoice_id": "TX-ST-204010"}, "valid_from": "2040-11-30",
    "valid_to": null}}

{"item_id": "fb-02-05", "decision": "reject",
  "finding_key": {"invoice_id": "TX-ST-204009", "class": "waived_but_billed"},
  "fact": {"type": "waiver_scope", "subject": {"person": "Cy Example", "party": "TBX-US"},
    "value": {"months": ["2040-10"]}, "evidence": "Confirmed by phone"},
  "scope": {"kind": "one_off", "applies_to": {"invoice_id": "TX-ST-204009"}, "valid_from": "2040-11-30",
    "valid_to": null}}

{"item_id": "fb-02-06", "decision": "resolve_conflict",
  "finding_key": {"invoice_id": "TX-US-2040-0602", "class": "overcharge"},
  "fact": {"type": "precedence", "subject": {"contract_id": "TBX-US-EXP-99", "lane": "USX1-USX2"},
    "value": {"use": "amendment", "amendment": "TBX-US-EXP-99/AMD-1"}, "evidence": "Agreed with the carrier"},
  "scope": {"kind": "standing", "applies_to": {"contract_id": "TBX-US-EXP-99", "lane": "USX1-USX2"},
    "valid_from": "2040-10-22", "valid_to": null}}

{"item_id": "fb-02-07", "decision": "override_amount",
  "finding_key": {"invoice_id": "TX-US-2040-0615", "class": "overcharge"},
  "fact": {"type": "dispute_closed", "subject": {"invoice_id": "TX-US-2040-0615"},
    "value": {"status": "settled", "amount_usd": "212.40"}, "evidence": "Settled with the carrier"},
  "scope": {"kind": "one_off", "applies_to": {"invoice_id": "TX-US-2040-0615"}, "valid_from": "2040-06-02",
    "valid_to": null}}

```

What each one means for your reports:

- confirm + sender_authority: the person has that authority for that contract and category from valid_from. A key that was cannot_determine for want of it can become a finding (with learned_at);

another person with the same Schedule 1 wording is not covered.

- `close + dispute_closed`: stop raising that key. A new invoice with the same wrong rate is a new finding.
- `close + rate_accepted`: stop raising that key, and treat the named notice as the rate for that contract and lane inside the dates. A later invoice at that rate is fine; one billed above it is measured against it (with `learned_at`). Other lanes keep their own answer.
- `one_off_approval + waiver_scope`: the waiver covers the named months on that key only (with `learned_at`).
- `reject`: that key is not a finding. Drop it.
- `resolve_conflict + precedence`: for that contract and lane, from `valid_from`, use the named source, so a conflict there becomes a finding or goes away (with `learned_at`). A later signed amendment still beats it.
- `override_amount`: use `fact.value.amount_usd` as the amount of that key (with `learned_at`).

How to use it without going too far:

1. A `standing` fact applies to a later key only if every filter in `applies_to` matches, inside its dates. The same person in another category doesn't match. The same lane on another contract doesn't match.
2. A `one_off` decision changes that one key and nothing else. Approving a waiver for March doesn't waive April. Applying it to April is one of the five mistakes that cap your score.
3. A feedback fact counts as level 2 from `decided_at`: every report after the feedback file uses it. What it covers is set by `scope`: documents and shipments dated inside `valid_from` to `valid_to`, even when they arrived before `decided_at`. A later signed document on the same subject beats it.
4. `close` stops that key only. A new invoice with the same wrong rate is a new finding.
5. Every item that changed because of feedback carries `learned_at`.

What to send us

Send it within 48 hours of getting the brief. Everything goes in your copy of this repo.

The list

1. **Your repo**, as a private GitHub repo or a zip. Tell us the commit hash: we grade that commit and nothing after it.
2. **SOLUTION.md**, your own write-up: how to run it, what works, what doesn't, and anything we should know before we run it. The template has a heading for each. Leave README.md, GLOSSARY.md and BRIEF.pdf as they are: they're the brief, and you keep them.
3. **answer_sheet.json** from `make sheet` (the exam mailbox), plus **report_card.txt** and **report_card.json** from `make grade` (the practice mailbox). Both come from your final commit. The answer sheet is pass or fail: its numbers have to match our key before we run your code on our set.
4. **evals/**: at least 10 cases of your own beyond our two samples, and **evals/results.json** from `make eval` with your harness's score on them. Tell us in SOLUTION.md which case caught a real bug.
5. **DESIGN.md**, short enough to print on one side of A4 (about 800 words). It covers your fact model, how you detect conflicts, how you compute confidence, what goes to a person and what `autonomous` mode is allowed to do, how feedback changes later behaviour, what happens when new documents arrive, and your cost per 1,000 documents. The template has a heading for each.
6. **AI_USE.md**: the AI tools you used, what you used them for, and the prompt log.

How to send it

Reply to the email that sent you the brief. Either invite the GitHub account named in that email to your private repo (read access is enough), or attach a zip of the repo with its `.git` folder. Put the commit hash in the email.

Don't make the repo public. Don't include your API key anywhere in it.

The prompt log

We want the real log, trimmed only for secrets. Paste it into **AI_USE.md**, or put the exported files in `ai_log/` and link them from there. A chat export, your tool's session files or a plain copy and paste all work. If you wrote some parts without AI, say which. We read the log to see how you split the work between you and the tools. A tidy log earns nothing extra, and a missing one costs you.

What `make validate` checks

Run it before you send. It prints one line per check, and exits 0 only when all of them pass:

- the harness answers `ingest`, `report` and `feedback` with the arguments the grader passes;
- every section of **SOLUTION.md** is filled in;
- **report_card.json** exists and came from `make grade`;
- **answer_sheet.json** exists, came from `make sheet`, and is dated on the exam mailbox's last report date;
- **out/exam_report.json** (written by `make sheet`) matches the schema, and every quote in it is on the cited page;
- **evals/cases/** has at least 10 cases besides the samples, and **evals/results.json** exists;
- every section of **DESIGN.md** is filled in, and it's under about 1,000 words;
- **AI_USE.md** names your tools and has a prompt log, or links to one in `ai_log/`;
- nothing in the repo looks like an Anthropic API key.

A green `make validate` means we can grade your submission. How well it scores is up to the grader and to us.